

Contents

1	XCore & FMML^X	3
2	Communication between Java and XMF	5
2.1	Example	5
2.2	Basics	5
2.3	Initiating an Action	6
2.4	Diagram Actions	7
2.5	Gathering information from the user	7
2.6	Diagram Communicator	8
2.7	Diagram Client	9
3	Instance Wizard	11
3.1	Basics	11
3.2	Generation	12
3.2.1	Probablistic (Boolean)	12
3.2.2	Weighed Generator (Enum)	12
3.2.3	Equal distribution (Integer & Float)	12
3.2.4	Expression (any)	12
3.2.5	Gaussian (float)	13
3.2.6	Pick from list (any)	13
4	Concrete Syntax Wizard	15

Chapter 1

XCore & FMML^X

1.1 Misc

1.1.1 Evaluating an Expression from a String

See listing ?? . The string in `text` is first parsed and then evaluated. If this succeeds, a sequence of 2 is returned, with the value at position 0 and null at 1. If this fails, the sequence has null at position 0 and an error message at position 1. A such one can check `at(1) = null`, whether it was a success.

```
try
  let
    exp = OCL::OCL.grammar.parseString(text,"Exp1",Seq{XOCL})
  then
    value = exp.eval(self,
                     Env::NullEnv(),
                     if env = null
                       then Seq{XCore,Root,Auxiliary}
                       else Seq{env,XCore,Root,Auxiliary}
                     end)
  in
    [value, null]
  end
catch(exception)
  [null, exception.toString()]
end
```

Listing 1: current implementation of the generator in java

Chapter 2

Communication between Java and XMF

2.1 Example

Assume the user wants to change a slot value from 0 to 1 (Integer). First the user initiates the action. This may be through a context menu, or through an action on the diagram. Then a dialog will open where the user enters the desired value. Then the dialog will check the data and if possible send it to XMF. To do so, the data must be in a simple format, such as String, Integer, Float, Boolean or Vector. In XMF the received data is interpreted, i.e. finding the actual class matching the class name/path in the string. A “manipulator”, (factory pattern) will check whether the action can be performed. A list of errors is returned, if any, and sent to Java, where a message dialog will show. After sending the request to XMF, usually another request is sent which asks for the current state of the model, i.e. an update. The model content is received by the Java side, which then updates the view.

2.2 Basics

In XMF there is the package FMMLx, which contains the meta-model. Once a model is created, it is created (as a package) in XMF, containing instances of the classes of the metamodel.

In Java there are also classes describing the FMMLx meta model. They mostly mirror the XMF classes, and for every diagram, the instances in XMF are mirrored by the instances in Java. There is redundancy between the

meta-classes, i.e. the java classes need to be adapted, when the XMF classes change. This disadvantage is in contrast to the advantages, namely having semantically rich classes on the java side, which helps designing the dialogs in JavaFX. Also, if the meta-model classes were not mirrored, then the JavaFX would alternatively need to be mirrored in XMF instead. Also all layouting needed to be mirrored in XMF. Furthermore the communication would be about graphical elements instead of classes, instances and properties. The java side would be completele ignorant of FMML^X and instead would only know nodes, edges and their sub-elements.

The instances, albeit being mirrored do not have redundancy. The instances in java are “single use” and (somehow) immutable. If an update is requested, the entire model is discarded and a new model is loaded. This avoids redundancy, but as a drawback the user has to presented with continuity. Therefore a “mapping” exists in XMF, which stores the state of the diagram. This includes amongst others the position and visibility of elements. The instances are updated (i.e replaced) in demand. This is done regularly after sending a change request. The update button also request an update without, in that case without sending a change request. It is assumed this is enough to keep the view in sync with the model. It is possible to edit the model in other ways without triggering an update, mut the average user is discouraged to do so. As such, the view may be out of sync with the model, but the danger is negligible, as the change requests have thorough checks in XMF before actually performing.

2.3 Initiating an Action

There are several ways to initiate a change.

- A double click on the canvas on a certain element. The class `tool.clients.fmm-lxd diagrams.graphics.NodeElement` has an `Attribute` action, which is a lambda expression. The `NodeElement` is usually created during the layouting process and is found in classes such as `tool.clients.fmm-lxd diagrams.DefaultFmmlxObjectDisplay`
- A menu item in the context menu. There is a package `tool.clients.fmm-lxd diagrams.menus`, which contains the default menu and other specific context menus. Which menu shows up depends on the selected item.
- A menu item in the diagram menu. See `tool.clients.fmm-lxd diagrams.fmm-lxd diagram.diagramViewComponents.DiagramViewHeadToolBar`.

- Clicking while a palette item is selected. Quite a complicated process, as for associations two clicks are required. In any case, the diagram processes the first and if necessary second click and if reasonable, initiates an action.
- A key stroke, typing “DEL” triggers the remove dialog (see `tool.clients.fmmlxdialogs.fmmlxdialog.diagramViewComponents.DiagramViewPane`)

2.4 Diagram Actions

In this class (`tool.clients.fmmlxdialogs.DiagramActions`), all possible model changes pass through. This is to avoid redundancy, in case there is more than one way to initiate an action. There is one such instance for each diagram.

This class controls the flow of the actions. First a dialog is initialized. Then it is shown, returning the user input, if closed with OK. If the input is valid, the request is sent through the Diagram Communicator. Finally an update trigger is sent to the diagram.

This structure is not predefined by the class, but instead must be adhered to in each new action individually.

2.5 Gathering information from the user

In some cases there is no additional data necessary. This step is then skipped.

As defined in `DiagramActions`, the relevant dialog is initiated and shown. These dialogs are found in various packages, the easiest way to find them is through `DiagramActions`. The dialogs use an empty JavaFX dialog onto which several user defined elements are added. If only one String is needed, a simpler standard String input dialog may be used.

Once the dialog is closed, an object is returned to the caller. If OK was pressed, then an instance of an inner class (like a struct in other languages) is returned, otherwise null.

However the most common type of inputs are strings, which aren't always what is needed. For Boolean, checkboxes may be used, for Integers spinners or sliders are an option. However in most cases, the input is gathered as a string. To avoid checking back and forth with XMF, or mirroring a parser, the string is sent to XMF to be evaluated there. As such, the java side does not check anything else. (In some cases a simple integer parser may check, whether expression likely to evaluate to an integer has entered. For strings,

this may be confusing. In order to get a string as evaluation result, the string has to be in quotes. To avoid confusion, the input field usually has a checkbox “isExpression”, which is set by default, except where strings are expected. Furthermore it cannot be deactivated in other cases. So for string the default option is “not an expression” allowing to enter the string without for quotes. A usecase for using an expression for a string may be where the string is not typed directly, but had been stored in a global variable before.

As such, String, Boolean, Integer and Float are covered, as well as Sets and Sequences thereof. It must be noted that some special Float values (NaN, Infinity) or really big integers may have trouble being parsed. For enums, a drop down menu / combobox is used. For other data types, which are not necessarily known at the time the dialogs were designed, an expression must be supplied which should evaluate to an instance of that type. That may be the constructor, or a global variable, which holds a value of the required type.

For date and currency, special inputs have been created. If a new one is to be created, it must be able to return a string, which is the expression to be evaluated in xmf to create or access the desired value.

2.6 Diagram Communicator

This class in the last step in java before sending the request to xmf, and also the first one for arriving data. This class also sets up the connection for the diagrams and creates new diagrams.

For each request, a “data-telegram” is created. This contains the name of the operation in XMF as well as the arguments. The arguments can only be of type string, integer, float, boolean and array. The name of the operation and the number of arguments must match a definition in `Clients::Fmmlx-Diagram::FmmlxDiagramClient`. However, not that the XMF operation has one more argument containing the request id. This extra argument is added shortly before the data is sent.

As the communication with XMF is asynchronous, those requests waiting for an answer, a return call has to be supplied, stating what shall be done once the response arrives. There are two layers of return calls here. One is the outer return call, which is supplied in the arguments when invoking the operations in this class. The other is the inner return call which is put into a hash map, together with a unique request id. Once the answer arrives this inner return call is executed. This usually contains unwrapping the data received from XMF. at the end of the inner return call, the outer return call

should be invoked.

2.7 Diagram Client

In this XMF class, the data sent is unwrapped. The evaluation of strings into XMF values usually happens here. The unwrapped data is then in most cases sent to a Manipulator class. The operations are usually designed in a way that they have a variable containing a sequence of errors. This list is then sent back to java, in order to open a popup menu if an error is present. As such, the Manipulator classes **MUST** return a sequence of errors.

This should have been done by a more conventional exception handling, which might look more familiar and less confusing.

Chapter 3

Persistence

A model is made persistent in the form of an XML file.

3.1 Saving

To save a model, the user chooses the menu item or presses Ctrl+S, which creates an instance of `tool.helper.persistence.XMLCreator` and calls the operation `createAndSaveXMLRepresentation()`. This calls `FmmlxDiagramCommunicator.getAllDiagramInfos()`, which returns a list of diagrams. Then a dummy diagram is created, to fetch a list of imported models and the model data. The model data contains commands required to recreate the model. This is converted to xml. Then the diagram data for each diagram is fetched, including all node and edge positions, as well as settings such as “show derived attributes” and the current center of view.

3.2 Loading

In `FmmlxDiagramCommunicator`, `openXmlFile()` is called from XMF, as the menu item calls XMF first. A parser object (`tool.helper.persistence.XML-Parser`) is created, which first opens a dialog to choose a model file. The operation `parseXMLDocument` then opens imports the model. It first checks for other required models, at the moment only a text message is given if the required models are not yet present. Then, a dummy diagram is created whose only purpose is to import the models, not the diagrams yet. However this process requires a diagram to be present. To avoid different handling for the first and the subsequent diagrams. First the model is imported, and then all diagrams are imported.

To import the model, The operation `sendModelDataToXMF` sends a request for each item in the xml model similar to those when using the diagram. So a class is added by sending the same command to XMF, as if the user had clicked on the menu item for adding a class. The same is done for subclassing, attributes, slot values etc.

Then for each diagram, the operation `buildDiagram()` sends a command for each node (objects) and for each edge (associations, links, etc.) with their coordinates, and whether they are shown or not. This is the same command called when an object or edge is moved on the screen.

Chapter 4

Instance Wizard

The instance wizard creates a number of instances of a given class on the next level below. The user needs to set the following options:

- the number of instances to be created
- the methods for generating value for each attribute
- constraints which the generated instance needs to conform to
- the maximum number of failed attempts for each instance to be created (see below)

4.1 Basics

Once approved, the wizard will generate a set of values. The randomization may happen in java, or through an instruction to xmf invoking a random generator there. The generated instance will be checked for the selected constraints. If any constraint is violated, the attempt fails and a new set of values is generated. If the number of maximum failed attempts is reached because the constraints are unlikely or impossible, the wizard stops (or continues with the next instance, needs to be checked in detail) If there are no constraints selected, or all checks pass, the instance is accepted and added.

The point of the constraints is to add individual bounds for the values like for sizes or weights where the default random generator may be too broad in its range or for correlations between values, like gender and name.

4.2 Generation

4.2.1 Probablistic (Boolean)

Initially set to 50:50, the slider can be set to any likelihood for true and false respectively. 0.0 or 1.0 are the bounds which always yield the respective values. This is evaluated by java.

4.2.2 Weighed Generator (Enum)

A table is created with all enum items in the left column and their weights (initially set to 1) in the right column. The weights can be set to any non-negative value. The probability of each enum item is determined by its weight divided by the sum of all weights. As such, the weights do not need to add to a specific value. An enum with weight 0 is impossible to be chosen.

4.2.3 Equal distribution (Integer & Float)

A number is chosen from either a continuum or from a discrete set of consecutive number. A minimum and maximum needs to be supplied. In contrast to usual conventions, the maximum is inclusive. The numbers have an equal distribution, i.e. all integers between min and max are equally likely to be picked, similar for floats, as in java's `Math.random()`. This is evaluated in XMF.

However, there might be a bug effective here. The XModeler implementation for `Integer::random()` uses a random float between 0.0 and 1.0, then scales it up and rounds to the nearest integer. However, the correct implementation would be to always round down. As such, this generator may need to be repaired.

4.2.4 Expression (any)

The user must supply an expression, which is then each time evaluated in a global context in XMF. That means, no previously set slot values from the same instance may be used, which anyway is impossible as the order of value generation is not determined.

If the expression does not contain any random elements, the value will be the same for all instances.

4.2.5 Gaussian (float)

This generates a float from a Gaussian distribution. The mean and standard deviation need to be supplied. The range is not bound and any number may be generated. To generate the number internally, a built-in java functionality for mean 0.0 and standard deviation 1.0 is used and converted to the required range.

4.2.6 Pick from list (any)

The user supplies an expression which evaluates to a list. This may be created through the expression or it could be one previously prepared such as `Root::Auxiliary::maleNames`, `Root::Auxiliary::femaleNames` or `Root::Auxiliary::surnames`.

This expression is sent to XMF and (hopefully, depending on the skill of the user) evaluated to a list, then a random element is picked.

There may be the same bug as before, skewing the probabilities. This needs to be investigated.

Also the list is sent twice, see code fragment. Either another way of picking a random element must be found, or a let-structure must be used.

```
list + ".at(Integer::random(" + list + ".size()-1))";
```

Listing 2: current implementation of the generator in java

Chapter 5

Concrete Syntax Wizard